

The first thing is that a function can have multiple names

```
• def hello(str):
    print(str)
• hi=hello
• hello("greetings")
• hi("greetings")
```

The output for calling either hello or hi is the same i.e. “Greeting”. Deleting either one of the function names will not delete the function name itself. Next you need to understand nested functions. Nested functions are functions defined inside another function. As shown here: <https://dgit.in/nestedPy>.

The two functions- `first_child` and `second_child`- are nested inside the parent function and cannot be called separately. If you do, you will get the error

```
• NameError: name 'first_child' is not defined.
```

So the two child functions are part of the parent function as a package deal. Take a look at the example here: <https://dgit.in/nestedPy2>.

What do you think the output will be? We have taken a function, written nested functions, written one more function and then modified the behaviour of the original function. So the output will be

```
• Read Code
• Understand Code
• Execute Code
```

`Read Code` is the nested function, `wrapper`, which is executed when the parent function, `learn_python` is called. Then the function is modified with another function `using_ft` to print `Understand Code` and finally it prints the original parent function parameter coding which gives the result `Execute Code`. Put simply, decorators wrap a function, modifying its behaviour.

You can substitute `using_ft = learn_python(using_ft)` by using the `@` symbol to call the decorator. This is called the “pie” syntax. So to call the decorator, the code would read as shown here: <https://dgit.in/piesynPy>.

This would give the same output as the previous example.

One of the most used decorator functions is the `login_required()` decorator, which ensures that a user is logged in/properly authenticated before they can access a specific route (`/secret`, in this case, as show here): <https://dgit.in/loginPy>.

Now let's take a look at class decorators. A decorator is simply a callable object that takes a function as an input parameter. A callable object is an object which can be used and behaves like a function but might not be a function. It is possible to define classes in a way that the instances will be callable objects.